DAA Assignment 4 REPORT

Group: SE-2525

Khasan Abi-Mansur

Experiment Design Card

Research questions

How do input size and input type affect the work and memory usage of recursive, full-table, and two-row Edit Distance?

How does KMP compare with Naive search as the text and pattern grow, especially on repetitive inputs?

Input sizes

Edit Distance. All three algorithms use equal-length strings with n = 3, 6, 9, 12. The full-table and two-row versions also use n = 128, 256, 512, 1024, 2048. The smaller sizes keep recursion practical, while the larger sizes show how dynamic programming scales.

String Search. The text length is n = 4096, 8192, 16384, 32768, 65536. The pattern length is m = n / 64, so the pattern grows with the text.

Input types

Edit Distance. Identical strings show the short path through recursion. Random strings over ABCD represent mixed inputs. Disjoint strings made entirely of A and B, respectively, force recursion to explore all three choices at every mismatch.

String Search. Random text contains a pattern planted at n / 3 and 2n / 3. The adversarial case uses an all-`A` text and a pattern of m − 1 copies of A followed by B, forcing Naive to fail near the end of the pattern at each position. The immediate-mismatch case uses an all-`B` text and an all-`A` pattern.

Metrics and protocol

I use the fixed seed 20261108, two warm-up runs, and five measured runs per case. I save each run in results.csv and compare median times in milliseconds. For Edit Distance, I count recursive calls or evaluated table cells, character comparisons, and peak integer-array slots. For string search, I count text comparisons and include LPS construction in KMP's comparison count and time.

Predictions before running

I expect recursion to grow exponentially on disjoint strings and linearly on identical strings. Both dynamic programming versions should do the same amount of work, but the two-row version should use less memory. On the adversarial search input, I expect doubling both lengths to roughly quadruple Naive's comparisons and double KMP's.

Planned plots

1. Edit Distance state evaluations versus string length, with a logarithmic vertical axis.

2. Full-table versus two-row storage in integer slots, with logarithmic axes.

3. Naive versus KMP character comparisons across input types, with logarithmic axes002E

4. Median Naive and KMP running times on the adversarial input, with logarithmic axes.

Prediction vs Result

Edit Distance. I expected plain recursion to grow quickly on disjoint strings because it repeatedly solves the same sub problems. At n = 12, recursion made 377,393,953 calls for disjoint strings and only 13 calls for identical strings. Both dynamic programming versions evaluated the same number of states. At n = 2048, the full table used 4,198,401 integer slots, while the two-row version used 4,098, confirming the predicted memory savings.

String Search. I expected Naive to repeat many comparisons on repetitive text and KMP to reuse information from its LPS array. On the adversarial input, doubling the text and pattern lengths roughly quadrupled Naive's comparison count and doubled KMP's. The timing trend was less regular. For example, KMP's median fell from 0.1380 ms at n = 4096 to 0.0402 ms at n = 8192 despite doing more comparisons. JVM warm-up and scheduling may explain this difference.

Short Answers

1. Which input made your string search slowest, and why?

The slowest case was Naive search on the adversarial input with n = 65,536 and m = 1,024. The text consisted entirely of A characters, and the pattern contained 1,023 A characters followed by B. At each possible starting position, Naive matched almost the entire pattern before failing on the last character. It made **66,061,312 comparisons**, with a median time of **124.648 ms**.

2. How large an n could recursive Edit Distance handle, and what stopped it?

The largest size I tested was n = m = 12. On disjoint strings, it completed in a median of **693.2245 ms** and made **377,393,953 recursive calls**. At n = 9, the same input type required only **2,193,844 calls**. I capped the experiment at 12 to keep repeated runs practical as the number of calls grew.

3. What do you gain and lose when you use two rows?

Using two rows reduces the extra memory from **O(nm)** to **O(m)** while keeping the same distance and **O(nm)** running time. At n = m = 2048, storage dropped from **4,198,401** to **4,098** integer slots. Earlier rows are overwritten, so the resulting arrays cannot be used to trace back through the full table and recover an edit sequence directly.

4. Which string search would you choose for the city platform, and why?

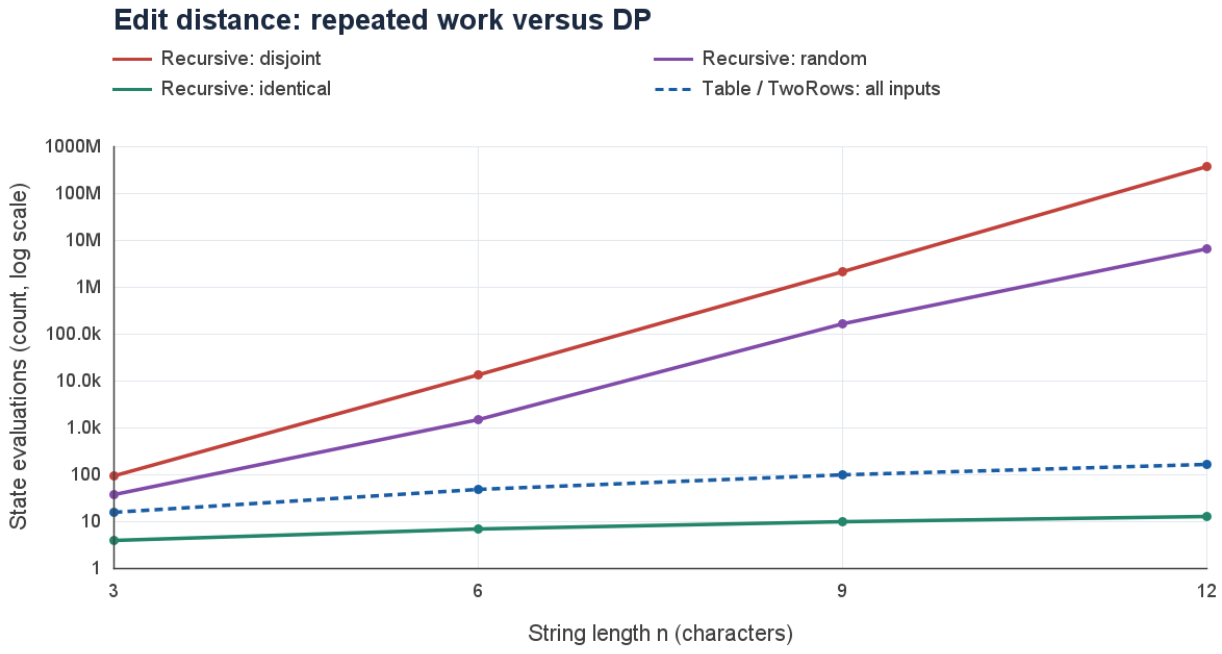
I would choose **KMP** for exact searches in long city logs. Its **O(n + m)** running time keeps repetitive input from causing the repeated work seen in Naive search. On the largest adversarial case, KMP made **132,094 comparisons** including LPS preprocessing, compared with **66,061,312** for Naive. I would still use Edit Distance for matching misspelled street names.

One Paragraph on Time

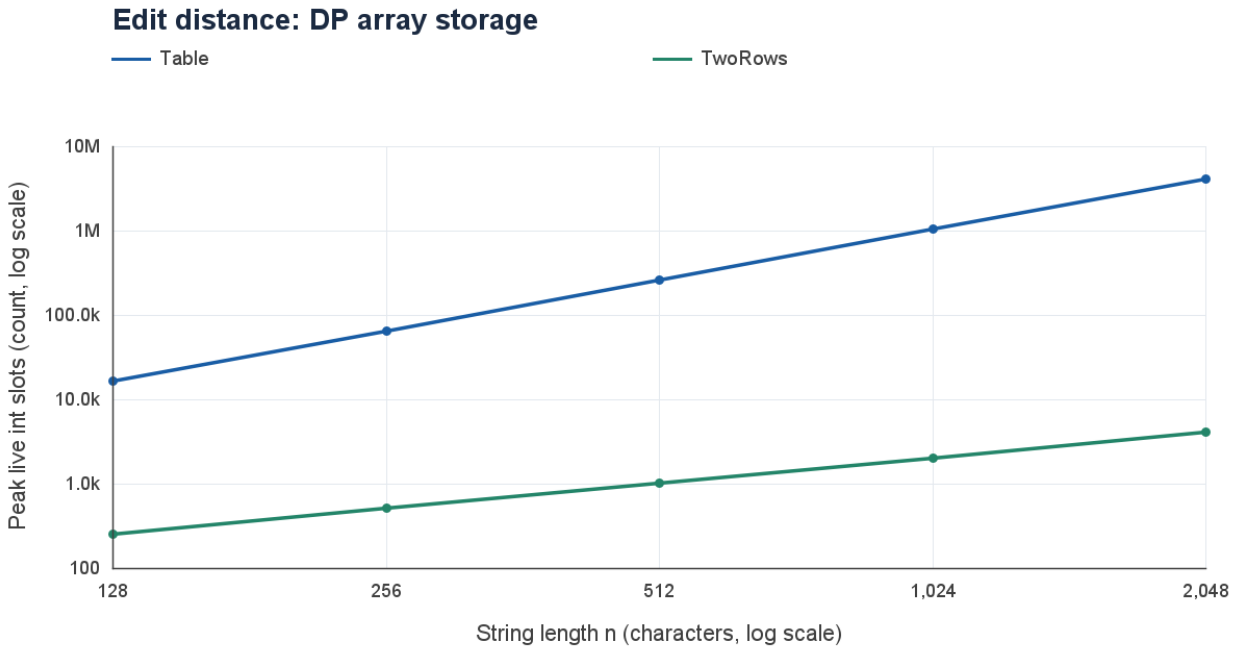
I used the median of five runs for each case. On the largest adversarial input, the median was **124.648 ms** for Naive and **0.3228 ms** for KMP. Naive's individual runs on that input ranged from **121.6765 ms to 126.7644 ms**, showing that the same work can take different amounts of time. JVM warm-up, garbage collection, and operating-system scheduling can all affect timings. I also used operation counts because they show how the amount of work grows with input size.

Plots

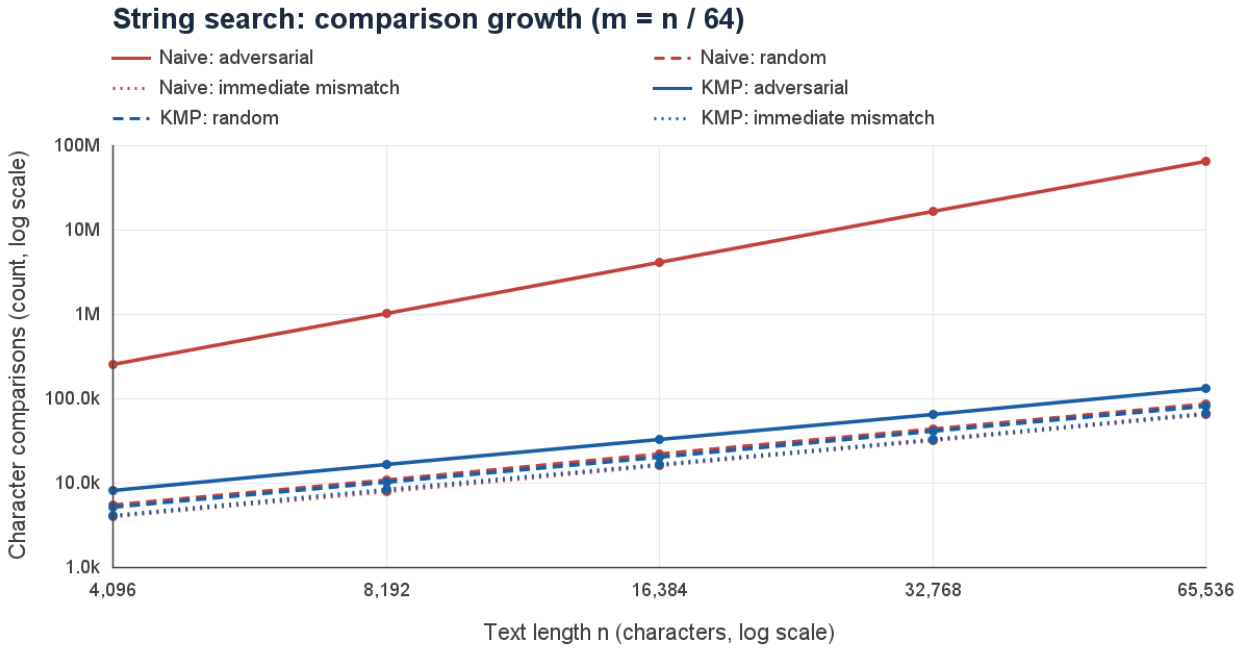
Edit Distance - number of computed states



2. Edit Distance - Memory Usage



3. Naive and KMP — number of comparisons



4. Naive and KMP - median search time on complex input

